ENDG 510 L01 - (FALL 2026)

Lab Manual

Instructor: Dr. Hadis Karimipour

Lab Tasks

TASK 1: sensor data acquisition in Cyber-physical System (CPS)

Objective: To understand the data collection process from CPSs and its subsequent storage for future analysis.

Required Components:

- IoT Sensors (i.e., temperature and humidity sensors)
- Stick PC (i.e., raspberry pi 4)
- Jump wires (female to female)
- Wi-Fi router/connectivity.
- Edge server (Laptop) (**You need to bring your own laptop and mobile data for the experiment**).

Task details:

1. Begin by physically linking the available sensors, such as those used for measuring temperature and humidity, to the specified hardware platform(Raspberry Pi). (More details: https://www.raspberrypi.com/), as shown in Figure 1.
2. Next, start the data acquisition process from these sensors using the Raspberry Pi as your data hub.
3. Following the successful connection and data collection, the next step is to establish a wireless communication link, preferably utilizing Wi-Fi technology, with the nearest available edge server. The edge server, for your convenience, can be any suitable computing device such as a laptop or desktop computer. Once the wireless connection has been effectively configured, your primary objective is to seamlessly transmit the gathered sensor data from the Raspberry Pi to the designated edge server. To accomplish this, make use of socket programming techniques to facilitate the wireless data transfer.
4. Upon the successful transmission of the data to the edge server, the final phases of the task involve two critical actions. Firstly, ensure that the received data is stored on the edge server for future reference and analysis. Secondly, simultaneously display the received data in a real-time manner.



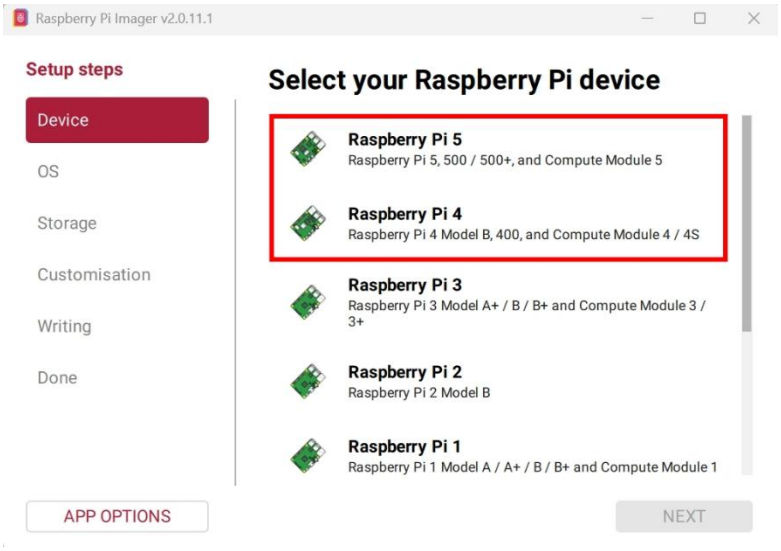
Figure 1 System design of the proposed task.

Procedure:

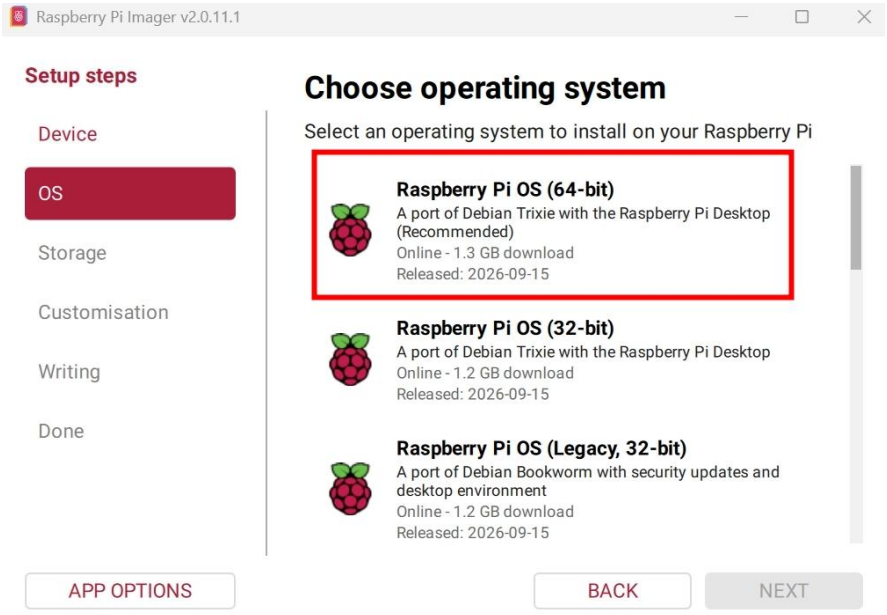
Prepare Raspberry pi:

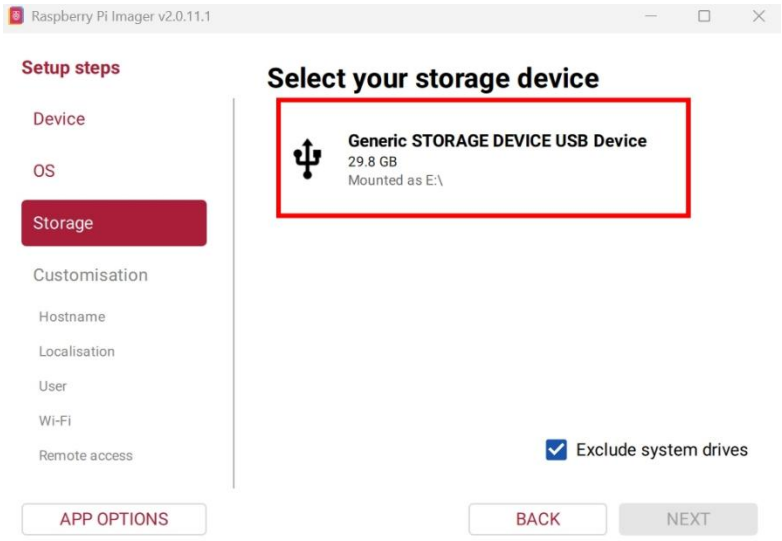
First install OS in the provided memory card as follows:

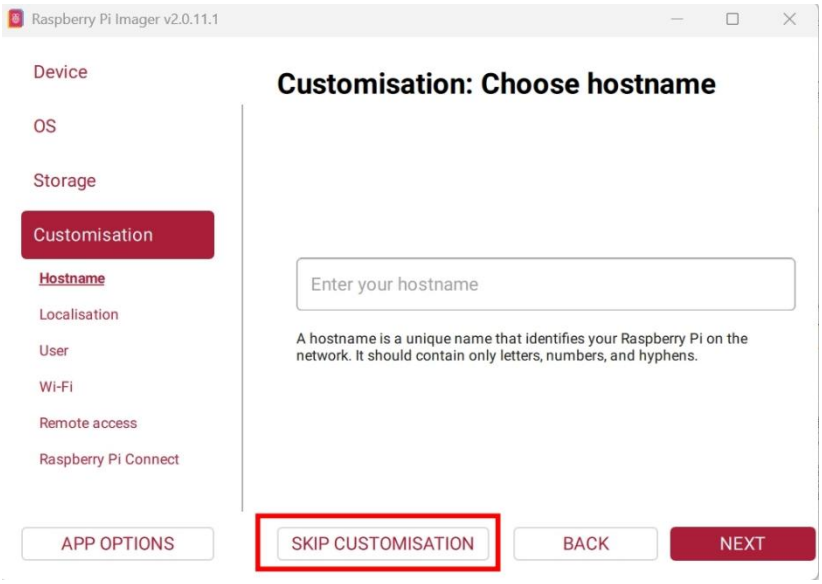
First download and install Raspberry Pi Image from this link: https://www.raspberrypi.com/software/. The view of the Raspberry Pi Imager will be as follows (if your board is pi 4 then choose 4 else choose 5):

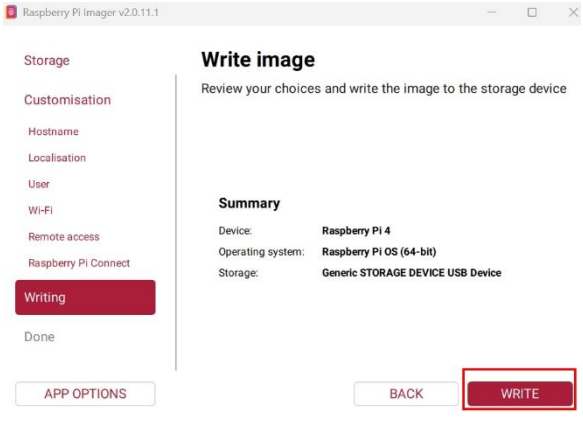


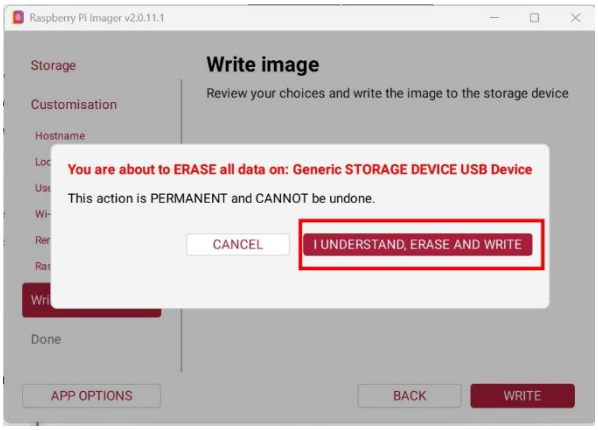
After that follow the following steps for installing OS follow the red marked options and press **next**:

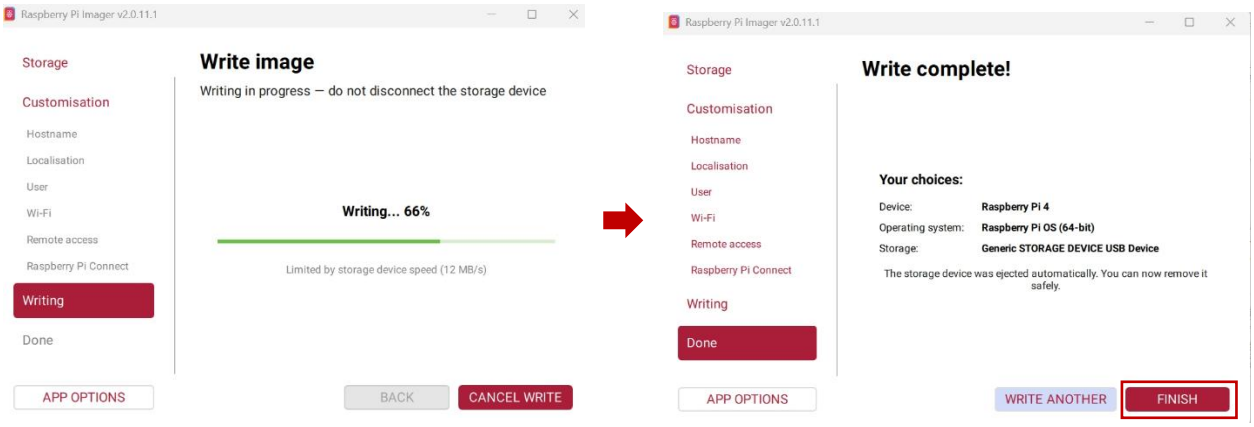



Raspberry Pi Imager v2.0.11.1
Setup steps
Device
OS
Storage
Customisation
Hostname
Localisation
User
Wi-Fi
Remote access
Select your storage device
Generic STORAGE DEVICE USB Device
29.8 GB
Mounted as E:\
Exclude system drives
APP OPTIONS
BACK
NEXT

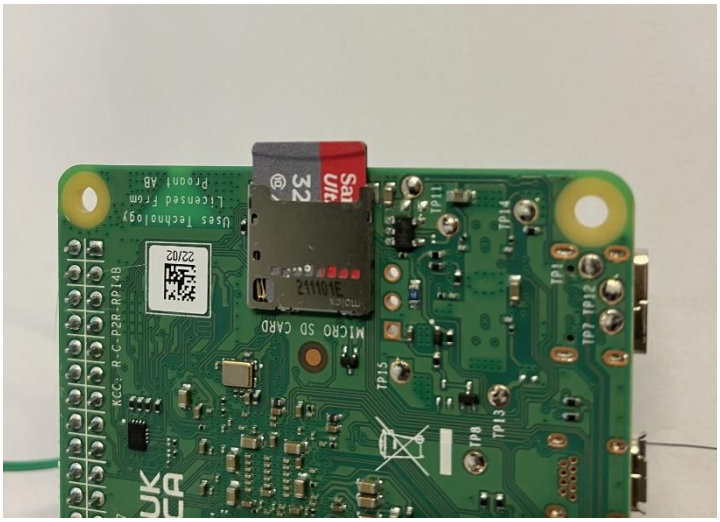

Raspberry Pi Imager v2.0.11.1
Device
OS
Storage
Customisation
Hostname
Localisation
User
Wi-Fi
Remote access
Raspberry Pi Connect
Customisation: Choose hostname
Enter your hostname
A hostname is a unique name that identifies your Raspberry Pi on the network. It should contain only letters, numbers, and hyphens.
APP OPTIONS
SKIP CUSTOMISATION
BACK
NEXT


Raspberry Pi Imager v2.0.11.1
Storage
Customisation
Hostname
Localisation
User
Wi-Fi
Remote access
Raspberry Pi Connect
Writing
Done
Write image
Review your choices and write the image to the storage device
Summary
Device: Raspberry Pi 4
Operating system: Raspberry Pi OS (64-bit)
Storage: Generic STORAGE DEVICE USB Device
APP OPTIONS
BACK
WRITE

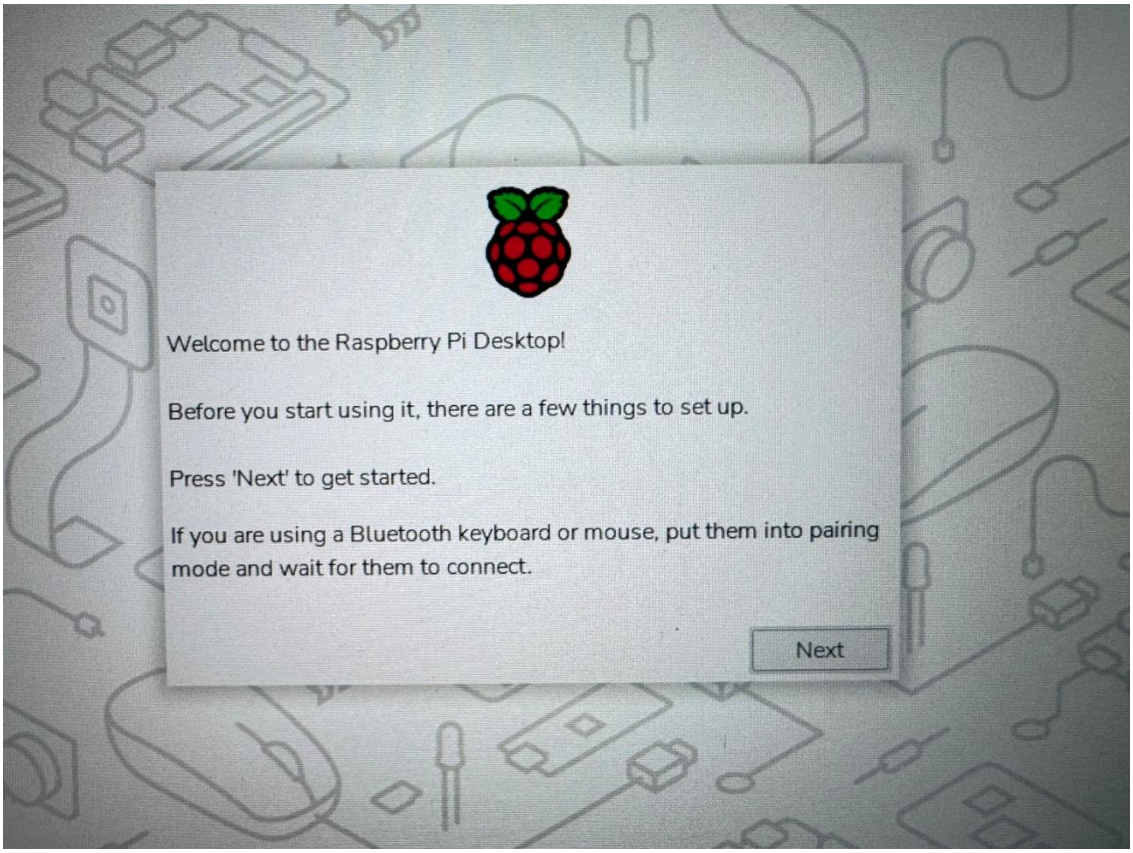

Raspberry Pi Imager v2.0.11.1
Storage
Customisation
Hostname
Done
Write image
Review your choices and write the image to the storage device
You are about to ERASE all data on: Generic STORAGE DEVICE USB Device
This action is PERMANENT and CANNOT be undone.
CANCEL
I UNDERSTAND, ERASE AND WRITE
APP OPTIONS
BACK
WRITE



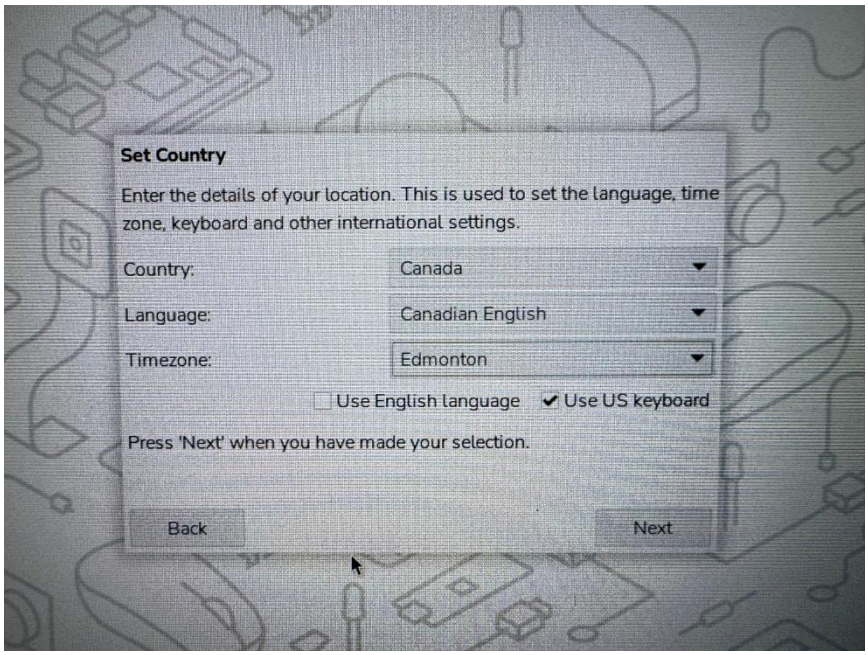
After that, wait for the installation to complete. Once the installation is finished, insert the memory card back into the Raspberry Pi as follows:



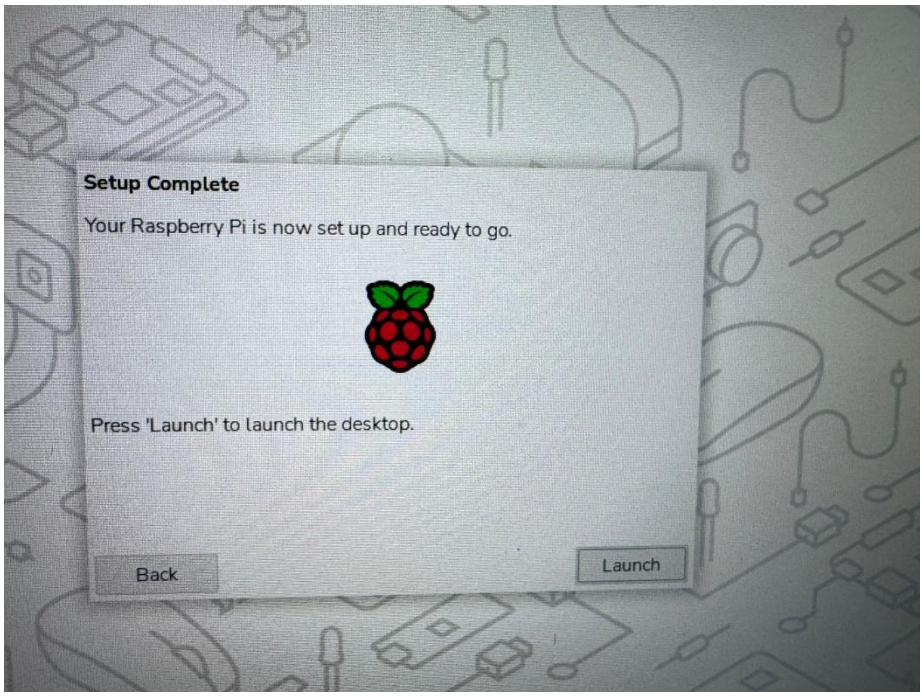
Once you start the raspberry pi, you might get the following screens to finish initial settings.



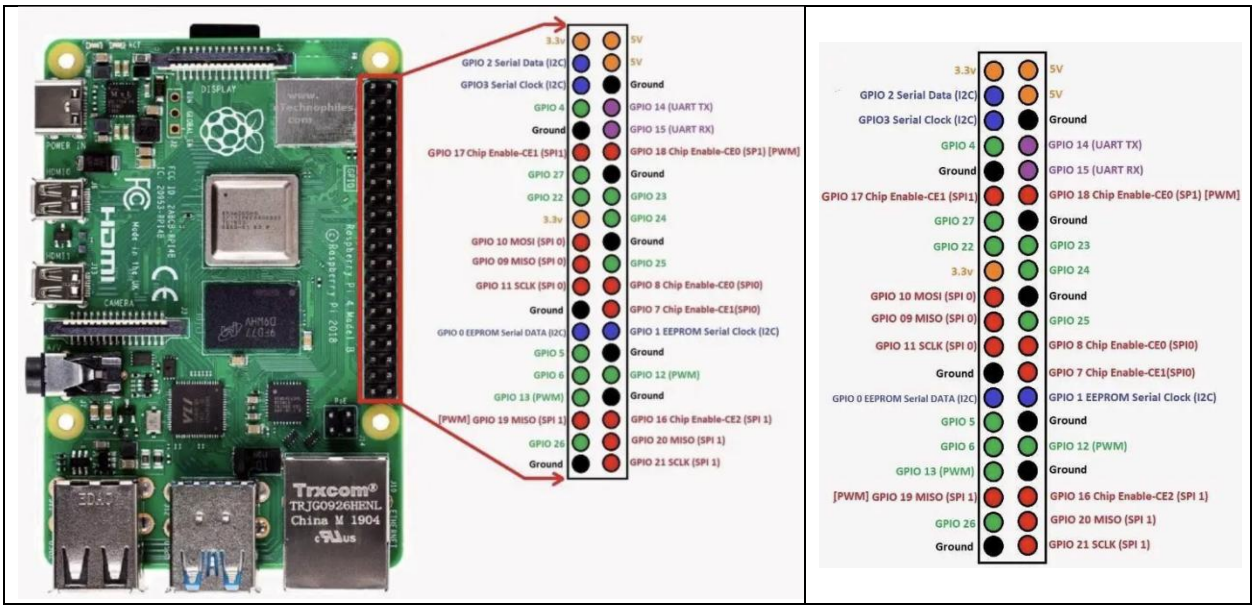
Choose suitable country (i.e., **Canada**), language (i.e., **English**), and timezone (i.e., **Edmonton**) in the following screen.



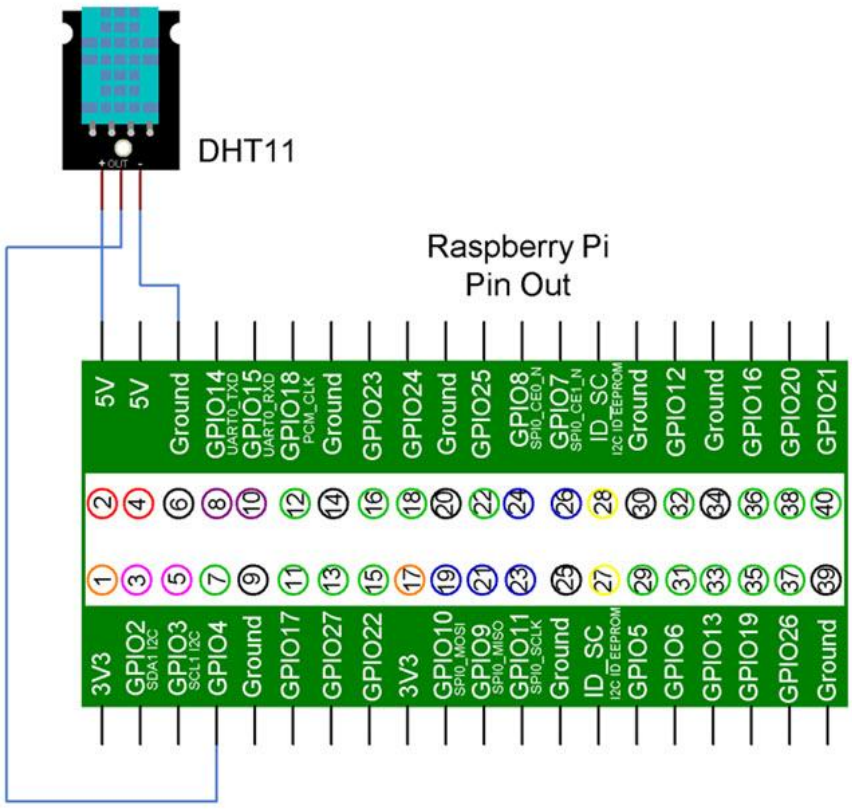
You can skip the password setup part. In case you are not allowed to skip the ID and password phase, enter **abc** as username and **abc** password that you can remember for later use. When wifi network prompted, enable your **mobile hotspot** and select that Wi-Fi network. Ensure that both the edge server (your laptop) and the stick PC (Raspberry Pi) are connected to the same Wi-Fi network. Keep Next until update software prompt arrives. After that click "Skip" until you reach the final step, as shown below.



Once the setup is complete, open the terminal (black screen; press **ctrl+alt+t**) on the Raspberry Pi and enter the following command, then press Enter: "**sudo timedatectl set-ntp true**" to adjust with the local time. After that type "**sudo apt-get update**" and press enter. It should run without any problems. If it responds with "OK" for everything, it means there are no errors. In case of an error, copy the error message and search for potential solutions on Google. If the "**sudo apt-get update**" command is successful, then proceed to enter the following command and press Enter: "**sudo apt-get upgrade**". It should also run without any issues. If it responds with "OK" for everything, it means there are no errors. In case of an error, copy the error message and search for potential solutions on Google. Once the Raspberry Pi is ready, you need to attach the sensor as shown in Figure 1. Connect the sensor to the Raspberry Pi using the provided jumper wires. To connect it to the Raspberry Pi, you must attach them to the **General Purpose Input/Output (GPIO)** pins. The description of GPIO pins is provided as follows.



After reading the GPIO pins, connect sensor with the raspberry by following the provided instructions.



After establishing the sensor connection with the Raspberry Pi, it's essential to verify and retrieve data from the sensor.

You'll be using the Adafruit DHT11 Python library.

To install the Adafruit DHT11 library:

Enter the following command:

sudo pip install adafruit-circuitpython-dht adafruit-blinka --break-system-packages

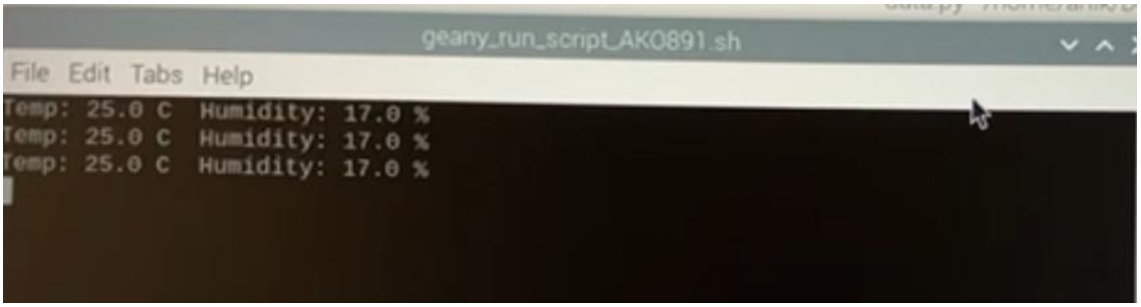
After that create an empty file named "test.py" and open it using **geany** (an IDE for the python). Following this, use the following code to get reading from sensor.

```
import time
import board
import adafruit_dht

# must use use_pulseio=False
dht = adafruit_dht.DHT11(board.D4, use_pulseio=False)

while True:
    try:
        temp = dht.temperature
        hum = dht.humidity
        print(f"Temp: {temp:.1f}°C   Humidity: {hum:.1f}%")
    except Exception as e:
        # Any error from the DHT read will be caught here
        print("Error reading DHT11:", e)
    time.sleep(2)
```

Once you run the code, you will get the following screen.



Afterward, you'll need to prepare your laptop to receive data from the Raspberry Pi. If your laptop doesn't have Python installed, you can visit https://python.land/installing-python to install Python. For convenient code management, we highly recommend using VSCode. You can download and install VSCode from here: https://code.visualstudio.com/download. Once your laptop is ready, you'll need to establish a Wi-Fi connection between your laptop and the Raspberry Pi, which will involve socket programming. To create a server and store data, you can use the following code. **Please note that the provided code will collect 10 readings**. To modify this value, refer to the comments within the code.

```
# first of all import the socket library

import socket
import pandas as pd # in case of error, install pnadas using: pip install pandas
# Create an empty Pandas DataFrame
df = pd.DataFrame(columns=['Temp', 'Humd', 'Label']) # Label: 1 means valid, 0
means invalid
# next create a socket object
s = socket.socket()
print ("Socket successfully created")
# reserve a port on your computer in our
# case it is 12345 but it can be anything
port = 12345
# Next bind to the port
# we have not typed any ip in the ip field
# instead we have inputted an empty string
# this makes the server listen to requests
# coming from other computers on the network
s.bind(('', port))
print ("socket binded to %s" %(port))

# put the socket into listening mode
s.listen(5)
print ("socket is listening")
# Establish connection with client.
c, addr = s.accept()
print ('Got connection from', addr)
# a forever loop until we interrupt it or
# an error occurs
i = 10 # only 10 reading will be taken. Increase it to higher according to you
plan.
while i>=0:
  #receive data from client
  client = c.recv(1024)
  data = client.decode()
  temp = data.split(" ")[0]
  hum = data.split(" ")[1]
  new_data = {
    'Temp': temp,
    'Humd': hum,
    'Label': 1
  }
  df = df._append(new_data, ignore_index=True)
  #print it
  print(data)
```

```
  i = i - 1
# Close the connection with the client
c.close()
#Export data to CSV
df.to_csv('data.csv', index=False)
```

To collect and transmit sensor data, put the following code in the raspberry pi. Replace **ip** with your laptop's **ip** address. To check how to find local **ip** address, visit this link: https://www.urtech.ca/2016/02/solved-how-to-determine-your-ip-address-on-windows-linux-solaris-mac-osx-and-the-internet/

```
# Import time library for delays
import time
# Import board library to access GPIO pins
import board
# Import Adafruit DHT library to read from DHT11 sensor
import adafruit_dht
# Import socket library for network communication
import socket
# Initialize DHT11 sensor on pin D4 with use_pulseio disabled
dht = adafruit_dht.DHT11(board.D4, use_pulseio=False)
# Create a new socket object
s = socket.socket()
# Connect to the server. Use your laptop's IP
s.connect(('172.20.10.4', 12345))
# Start an infinite loop to continuously read and send data
while True:
    try:
        # Read temperature from the sensor
        temp = dht.temperature
        # Read humidity from the sensor
        hum = dht.humidity
        # Print the values to the console in formatted style
        print(f"Temp: {temp:.1f}°C  Humidity: {hum:.1f}%")
        # Convert temperature and humidity to a string
        data = str(temp) + " " + str(hum)
        # Send the string data to the server through the socket
        s.send(data.encode())
        # Wait for 1 second before next reading
        time.sleep(1)
    # Catch any errors that happen while reading the sensor
    except Exception as e:
        # Print an error message if sensor reading fails
        print("Error reading DHT11:", e)
# Close the socket connection (will never reach here in infinite loop)
s.close()
```

You need to add fake data in the data.csv file as an attacker as follows. Try to insert as many as possible by changing values.

```
import pandas as pd # in case of error, install pnadas using: pip install pandas

# Read the CSV file into a DataFrame
df = pd.read_csv('data.csv')

new_data = {
    'Temp': 366,
    'Humd': 901,
    'Label': 0
}
df = df._append(new_data, ignore_index=True)

# Step 3: Save the DataFrame to a new CSV file
df.to_csv("data.csv", index=False)
```

Here, the **label '0 (i.e., invalid)'** indicates that the data has been improperly added to the dataset or database. On the other hand, **label '1 (i.e., valid)'** signifies that the data originates directly from the sensors. In the next task, you'll need to classify this data using machine learning. Therefore, it's important to store this data. To change the sensor reading, you can try wrapping your hand around the sensor and blowing on it. You'll observe changes in the values. Once you've created the data.csv file, store it (a minimum of 1000 data points is recommended) for the next task, which involves the classification process using machine learning.

Answer the following questions as part of your lab report.

1. What is a Raspberry Pi? Why did you use it? Which version did you use?
2. What is socket programming, a server, and a client?
3. Which protocol did it use in socket programming? What are the benefits of using that protocol, and what are other protocols that can also be used?
4. Which sensor did you use for the experiments, and what does it do? What are GPIO pins? What is "ground" in GPIO pins?
5. What do "encode" and "decode" mean in the provided code? Why are they used here?
6. Apart from socket programming, what are other ways to transmit data?
7. Why is 'Label' added in the data? What do 'valid' and 'invalid' mean? Which value represents 'valid' and 'invalid'?
8. What is an edge server and what are advantages of using an edge server?

TASK 2: Conduct data analysis on the gathered IoT data through the application of machine learning techniques.

Objective: To understand the application of machine learning for data analysis in CPS.

Required Components:

- Edge server (i.e., Laptop) (**You need to bring your laptop during the lab timing**)

Task details:

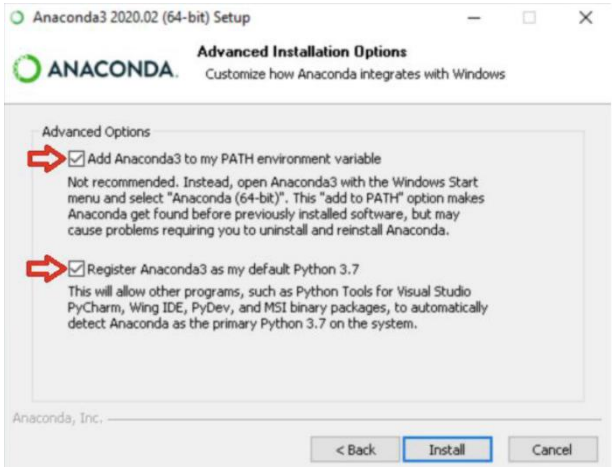
1. In the initial task, the data is endowed with labels, signifying a supervised learning approach for **Task 2**. To proceed, it is imperative to carry out **classification** based on the information acquired in **Task 1**. Before delving into the classification process, it is crucial to preprocess the gathered data, thoroughly scrutinizing for any potential missing values, null entries, or redundant data.
2. Subsequent to the preprocessing step, the dataset needs to be partitioned into distinct training and testing subsets. Following this, the data should undergo a standardization process, for instance, employing a technique such as standardization, to ensure uniformity across the dataset.
3. Once the scaling process is completed, the subsequent phase involves the implementation of suitable machine-learning algorithms to conduct the classification task.
4. Upon the completion of the training phase, it is imperative to evaluate the model's efficacy by subjecting it to the designated testing set. This evaluation process should culminate in the generation of a comprehensive performance report, encompassing various accuracy metrics for thorough analysis.
5. To conclude the process, it is recommended to save the trained model, facilitating its seamless deployment for future applications.

Procedure:

Initially, you should establish the experimental environment. To achieve this, you'll need to install Anaconda on your laptop.

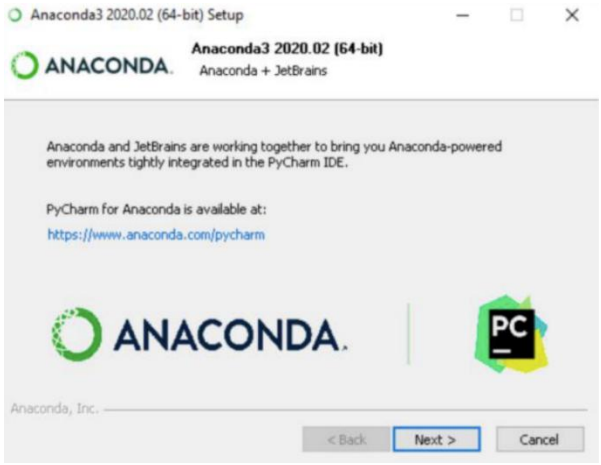
a- Windows (Reference: https://docs.anaconda.com/anaconda/install/windows/)
   1- Download Anaconda for Windows: https://www.anaconda.com/products/individual
   2- Double click on the installer to launch.
   3- Click “Next”.
   4- Read the licensing terms and click “I Agree”.
   5- Select an install for “Just Me”.
   6- Select a destination folder to install Anaconda and click Next.

7- In the Advanced Options, select BOTH options and click the Install button.



8- If you want to watch the packages Anaconda is installing, click Show Details.

9- Click the Next button



10- After a successful installation you will see the “Completing Anaconda3 Setup” dialog box. Click Finish.



11- To verify your installation, check this link: https://docs.anaconda.com/free/anaconda/install/verify-install/

b- Linux (Reference: https://docs.anaconda.com/anaconda/install/linux/)

1- 1- To use GUI packages with Linux, you will need to install the following extended dependencies for Qt (https://doc.qt.io/):

- Debian:

```
apt-get install libgl1-mesa-glx libegl1-
mesa libxrandr2 libxrandr2 libxss1 libxcursor1 libxcomposite1 libasound
2 libxi6 libxtst6
```

- RedHat:

```
yum install libXcomposite libXcursor libXi libXtst libXrandr alsa-
lib mesa-libEGL libXdamage mesa-libGL libXScrnSaver
```

- ArchLinux:

```
pacman -
Sy libxau libxi libxss libxtst libxcursor libxcomposite libxdamage libx
fixes libxrandr libxrender mesa-libgl alsa-lib libglvnd
```

- OpenSuse/SLES:

```
zypper install libXcomposite1 libXi6 libXext6 libXau6 libX11-
6 libXrandr2 libXrender1 libXss1 libXtst6 libXdamage1 libXcursor1 libxc
b1 libasound2 libX11-xcb1 Mesa-libGL1 Mesa-libEGL1
```

- Gentoo:

```
emerge x11-libs/libXau x11-libs/libxcb x11-libs/libX11 x11-
libs/libXext x11-libs/libXfixes x11-libs/libXrender x11-libs/libXi x11-
libs/libXcomposite x11-libs/libXrandr x11-libs/libXcursor x11-
libs/libXdamage x11-libs/libXScrnSaver x11-libs/libXtst media-
libs/alsa-lib media-libs/mesa
```

2- Download Anoconda for Linux: https://www.anaconda.com/products/individual#linux

3- Enter the following to install Anaconda:
bash ~/Downloads/Anaconda3-2020.02-Linux-x86_64.sh (**check the downloaded file name**). If you did not download to your Download directory, replace it with the path to the file you downloaded.

4- The installer prompts “In order to continue the installation process, please review the license agreement.” Click Enter to view license terms.

5- Scroll to the bottom of the license terms and enter “Yes” to agree.

6- The installer prompts you to click Enter to accept the default install location, CTRL-C to cancel the installation or specify an alternate installation directory. If you accept the default
install location, the installer displays “PREFIX=/home/<user>/anaconda<2 or 3>” and continues the installation. It may take a few minutes to complete.

7- The installer prompts “Do you wish the installer to initialize Anaconda3 by running conda

init?" respond "yes".

8- The installer finishes and displays "Thank you for installing Anaconda<2 or 3>!"

9- Close and open your terminal window for the installation to take effect, or you can enter the
command source ~/.bashrc

10- Verify your installation: https://docs.anaconda.com/anaconda/install/verify-install/

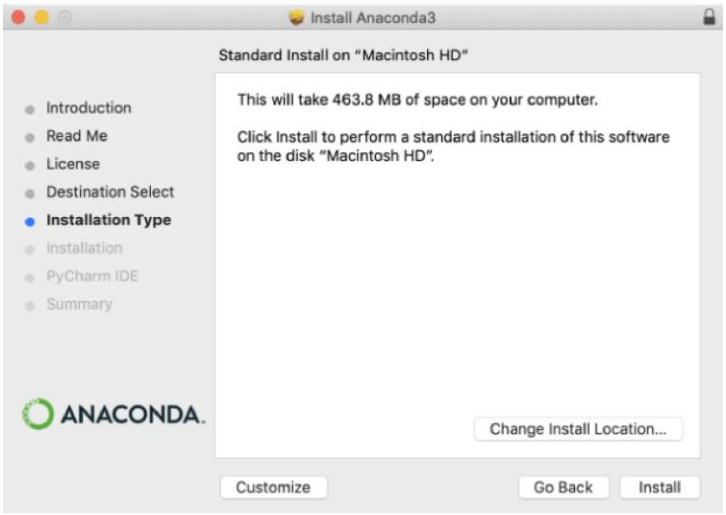
c- macOS (Reference: https://docs.anaconda.com/anaconda/install/mac-os/)

1- Download the graphical installer from: https://www.anaconda.com/downloads#macos

2- Double-click the downloaded file and click continue to start the installation.

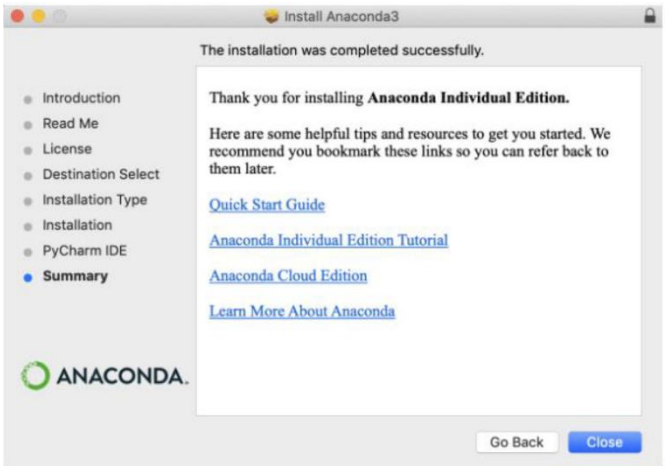
3- Answer the prompts on the Introduction, Read Me, and License screens.

4- Click the Install button to install Anaconda in your ~/opt directory (recommended)



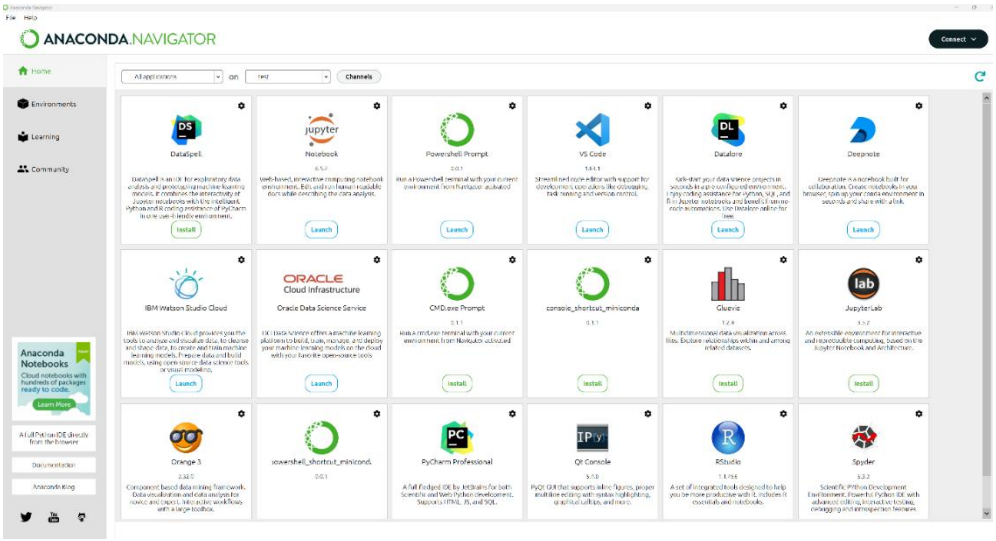
5- Click the continue button.

6- A successful installation displays the following screen

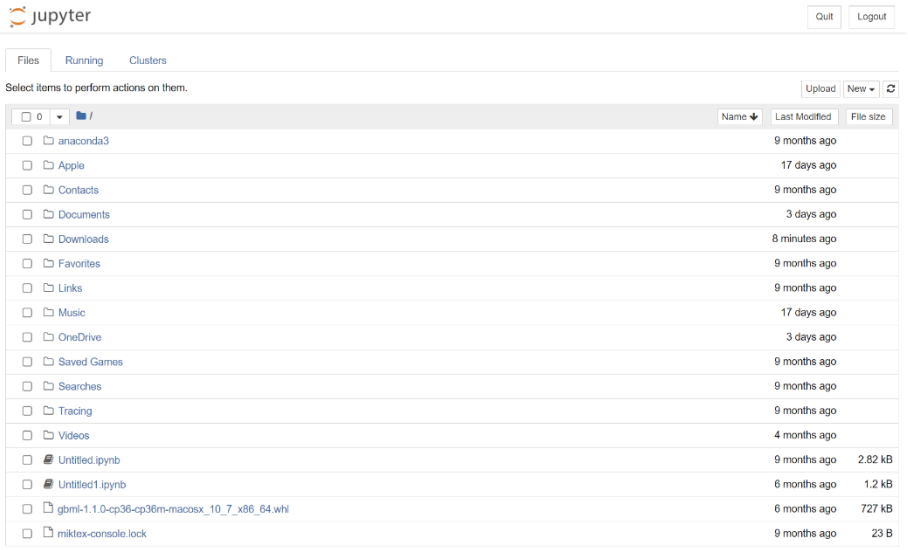


7- Verify your installation: https://docs.anaconda.com/anaconda/install/verify-install/

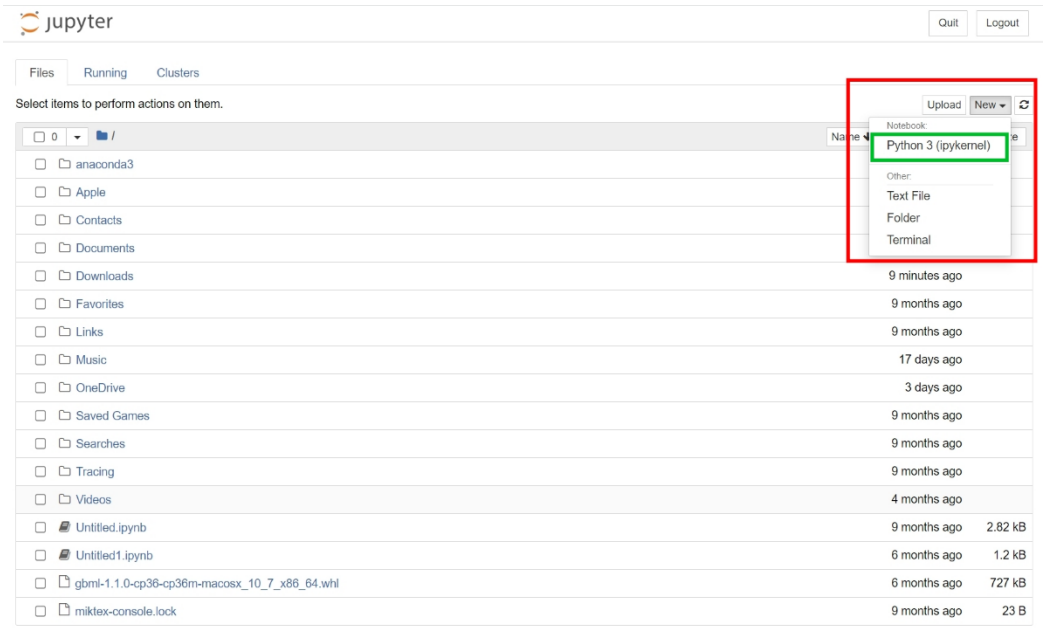
Once installed, open you anaconda and run jupyter notebook from that.



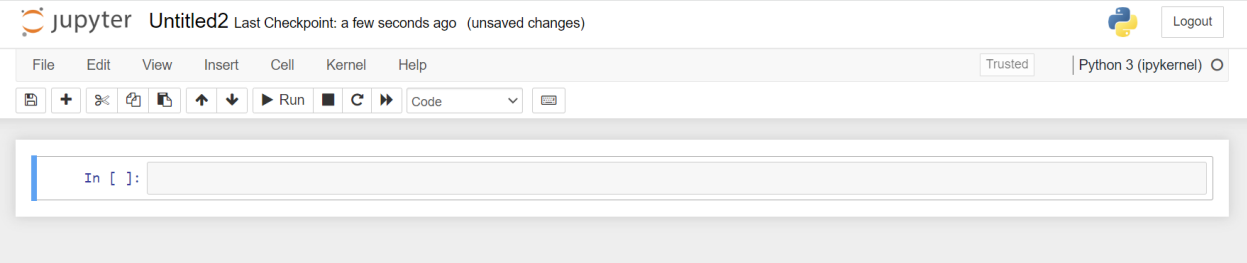
The view will be as follows once jupyter starts.



After that, to create new. ipynb (i.e., programming environment for ML) file, click on the new button (marked as red) from the top right corner of the screen and choose Python 3 or python 2 (marked as green) from the menu as follows.



The screen will be as follows:



Once that is loaded, first provide the following packages.

```
#load class
import numpy as np
import pandas as pd # in case it's not installed then install using conda create
-c conda-forge -n name_of_my_env python pandas
#       more       installations       information       can       be       found:
https://pandas.pydata.org/docs/getting_started/install.html
import os
#load sklearn module for creating and evaluating ML models. In case sklearn isn't
installed,
#follow this link https://scikit-learn.org/stable/install.html for further details
on the installation process.
from sklearn.neighbors import KNeighborsClassifier #load your classifier. In this
code, I have used KNN. You can choose other algorithms. You have to use at least
3 to complete this.
from sklearn.preprocessing import StandardScaler #module for perform scaling
```

```
from sklearn.model_selection import train_test_split #module for splitting datatset
from sklearn import metrics #module for evaluating performance
```

After that, load data from your .csv file as follows.

```
#load your data
df = pd.read_csv("data.csv") #change the name accordingly
df.head() # prints top 5 rows from the datatset to check data is load or not
```

To check missing values, follow the following code.

```
# Check for missing values
missing_values = df.isnull().sum()
print("Missing Values:\n", missing_values)
```

To remove duplicate values, follow the following code.

```
# remove duplicatesd
df = df.drop_duplicates()
```

After that, you need to prepare data as a feature space and label (i.e., class that will be used to classify) space as follows.

```
# prepare features
x = df.drop(['Label'],axis=1) #remove class or label
y = df['Label'] #load label
```

After that, you need to split your data into training set and test set as follows.

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=.2) #split datatset. Here ratio is 80:20. Change accordingly
```

Following this, you need to scale your data as below.

```
# Scale the data using standardization
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train) #scale training set
x_test = scaler.transform(x_test) #scale test set
```

After that select a classifier (i.e., ML algorithm) to perform training and test as follows.

```
z = KNeighborsClassifier(n_neighbors=3) # KNN classifier for 3 neighbours
KNN = z.fit(x_train,y_train) # start training
```

```
predict = KNN.predict(x_test) # performance in the test set
```

You can choose any algorithm. This algorithm isn't fixed. You can find more information on algorithm by following this link: https://scikit-learn.org/stable/supervised_learning.html#supervised-learning. Finally, check your performance by following this code.

```
print("Accuracy:", metrics.accuracy_score(y_test,predict)) # evaluating the performance based on accuracy
```

After that, store the model as follows.

```
# library for save and load scikit-learn models
import pickle
# file name, recommending *.pickle as a file extension
filename = "model.pickle"

# save model
pickle.dump(z, open(filename, "wb"))
```

You can choose any metrics. It's not fixed. Try to use at **least 3 algorithms** and make a performance table by comparing based on any performance metrics.

Answer the following questions as part of your lab report.

1. What is Anaconda? What benefits does Anaconda provide?
2. What is a missing value, null, or duplicate? Why do we need to clean them?
3. Why do we need to split our data into training and testing sets?
4. Why do we need scaling? What are the existing scaling techniques? What are the drawbacks of the min-max scaling technique?
5. Which machine-learning model did you choose? Why?
6. How did you evaluate your machine-learning model? Present your results in figure or table.

Format of the final report:

- **Introduction**
  - In this introduction, you will provide details about the task. For example, you will discuss what the task was about, how tasks were discussed among your group, how the task was divided, who did which tasks, and so on.
  - Which algorithms did you consider, and why did you consider them?
- **Experiments**
  - How did you perform the experiment (including expected screenshots)?
  - What difficulties did you face during the experiment, and how did you solve them?
  - How did teamwork assist you in completing the experiment?
- **Performance Evaluation**
  - Highlight the algorithms that you considered for the tasks and provide a comparative analysis, along with a figure or table to illustrate this.
- **Code & dataset**
  - Upload your code and dataset to a cloud and provide the link, ensuring that the link is active.

Overall, make sure to answer the provided questions, which are attached to each task, in their respective sections and ensure they are highlighted appropriately. Additionally, include each member's name and ID accurately, along with a description of their individual contributions.